

Mini Lesson: Lambda Functions

 <https://github.com/learn-co-curriculum/python-p3-lambdas>  <https://github.com/learn-co-curriculum/python-p3-lambdas/issues/new>

Learning Goals

- Create single-line, anonymous functions using the `lambda` keyword.
-

Key Vocab

- **Inheritance**: a tool that allows us to recycle code by creating a class that "inherits" the attributes and methods of a parent class.
 - **Composition**: a tool that enables you to recycle code by adding objects to other objects. Rather than building on a base class as in inheritance, composition leverages the attributes and methods of an instance of another class.
 - **Subclass**: a class that inherits from another class. Colloquially called a "child" class.
 - **Superclass**: a class that is inherited by another class. Colloquially called a "parent" class.
 - **Child**: another name for a subclass.
 - **Parent**: another name for a superclass.
 - `super()`: a built-in Python function that allows us to manipulate the attributes and methods of a superclass from the body of its subclass.
 - **Decorator**: syntax that allows us to add functionality to an object without modifying its structure.
-

Introduction

Defining functions and methods in Python typically requires use of the `def` keyword and a specific name that we can use to invoke it later on. This syntax allows us to specify many things like data types, default values, keyword arguments, and so on. That being said, it always requires us to use multiple lines of code. What if we're just trying to do something simple, like add 12 to a value for each new dozen eggs?

Python provides us an alternative for simple cases like this: the `lambda` keyword. If we wanted to add 12 to a value as mentioned above, we could write a lambda function like this:

```
new_dozen = lambda n: n + 12
```

When we invoke `new_dozen()` on a number `n`, our lambda function will add 12 to the total.

Lambda functions (or lambdas, for short) are small, anonymous functions that can take any number of arguments and manipulate them in any way that can be carried out in a single line of Python code.

Functions Are First Class Objects

It's important that you remember that functions are first class objects: they can be treated just as you might any local variable or instance around them.

While there are a number of cases where it makes more sense to define a lambda than write `n + 12` over and over again, the real power in lambdas comes from our ability to pass them as arguments to and return them from other functions.

Imagine we wanted to create a function that would add any specific number when invoked. We could write lambdas for each new number, or we could treat our lambda as a first class object and manipulate it inside of another function:

```
def myfunc(x):  
    return lambda n : n + x
```

```
new_century = myfunc(100)
```

```
print(new_century(2000))
```

```
# => 2100
```

- Anonymous means that an individual's name has been left out. why would we consider the lambda above to be anonymous?

When To Use Lambdas

Lambdas are typically used as arguments and return values. Certain built-in functions, like `sorted()`, take a function `key` as an argument to determine which values to sort by:

```
l = [['red', 'truck'], ['blue', 'truck'], ['red', 'sedan']]
sorted(l, key=lambda v: v[1])
# => [['red', 'sedan'], ['red', 'truck'], ['blue', 'truck']]
sorted(l, key=lambda v: v[1], reverse=True)
# => [['red', 'truck'], ['blue', 'truck'], ['red', 'sedan']]
```

This sorts our list of lists by the values at index 1 (the type of vehicle), with the added option to sort in ascending or descending (`reverse=True`) order.

We can also return lambdas as new, unique functions, as seen in the previous section:

```
def myfunc(x):
    return lambda a, b : (a + b) * x

sum_times_2 = myfunc(2)

print(sum_times_2(10, 20))

# => 60
```

Conclusion

Lambdas aren't the most commonly used tool in Python, but they can provide quick solutions when multi-line functions aren't necessary. The real strength in lambdas comes in passing them to and returning them from other functions, allowing for the creation and modification of simple functions without writing repetitive code. Remember your lambdas to stay DRY!

Resources

- [Python 3.8 Documentation](https://docs.python.org/3.8/)  (https://docs.python.org/3.8/)
- [Python Lambda - W3schools](https://www.w3schools.com/python/python_lambda.asp)  (https://www.w3schools.com/python/python_lambda.asp)